REFLECTION

Ethan Schweinsberg - 25267321

What's the worst case for your algorithm, and what kind of input triggers it?

The Karger-Stein's success rate is at its worst with graphs with exponentially many near-minimum cuts. In our benchmarking, the worst case was the random graph graph family. Because Karger-Stein relies on recursive branching and graph compaction, high edge density maximizes the data that must be scanned and sorted at every single node of the recursion tree. In this the algorithm made over 131,000 recursive calls. The O(ElogE) sorting step that merged parallel edges was bottlenecked by the CPU cache. This shows the algorithm's worst case is dense topologies where random contraction does not create a sparse sub problem.

What was the hardest bug or design decision you encountered, and how did you resolve it?

The hardest design decision was implementing the graph compaction step and having correct vertex mappings across the recursive frames. I struggled with how to translate the binary cut sides returned by a child call back into the parent graph's vertex space. At the start I passed sparse component IDs down the tree and the algorithm kept on processing O(E) sized arrays instead of properly shrinking to O(t^2) space.

To resolve this I used a root_to_compact lookup table. I mapped the current Disjoint-Set roots to a 0 to 1 -t index space before branching. This allowed me to build a edge list for the child frame and map the returning recursive side[] array back up the call stack to the original vertex space

If you did this implementation again from scratch, what would you do differently?

If I were to implement Karger-Stein again from scratch, I would change its memory architecture to operate in-place. To get the theoretical time bounds, my current implementation allocates new std::vector instances for edges and prefix weights at every step. This caused in 0.5 microseconds per call constant-factor overhead, which was spent on heap allocations. This is why standard Karger beat my implementation in raw wall-clock time despite Karger-Stein needing less runs. In a rewrite, I would pre-allocate a single global memory buffer for the recursion tree and pass array spans down the call stack.

What's one thing about your algorithm that you didn't fully understand before starting, and now do?

Before starting the project, I thought that because Karger-Stein's total bound of O(N^2log^3N) was mathematically better than to standard Karger's O(N^4logN), it would be faster in practice. Now I understand the impact of constant-factor overhead. I did not initially get that the recursive calls at N = 100 created a dense tree where the bottom layers only eliminate one vertex per level but still branch twice. I now know that high success rates and high costs are linked in this algorithm and that the recursion tree that makes the minimum cut is one that puts a bottleneck in the hardware level.